Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра систем штучного інтелекту



Звіт

до лабораторної роботи № 3

з дисципліни

Технології захисту інформації

Виконав:
студент КН-315
Лозовий Олег

Прийняв:
Федун В.І.

Львів – 2023

СТВОРЕННЯ ПРОГРАМНОГО ЗАСОБУ ДЛЯ ЗАБЕЗПЕЧЕННЯ КОНФІДЕНЦІЙНОСТІ ІНФОРМАЦІЇ

Мета роботи: ознайомитись з методами криптографічного забезпечення конфіденційності інформації, навчитись створювати комплексні програмні продукти для захисту інформації з використанням алгоритмів симетричного шифрування, хешування та генераторів псевдовипадкових чисел.

Хід роботи :

16.	16	20	8

Код програми:

```
from time import time
from lab1 import pnum
from md5f import MDFIVE


import os


w = 16 * 2
r = 20
init_vect = []
init_vect = pnum()[int(time())%150].to_bytes(w//8, byteorder='little')


def key_to_l(hashed):
    w2 = w // 2
    u = w2 // 8

    x = len(hashed)

    l = []
    for i in range(0, int(x), u):
        l.append(int.from_bytes(hashed[i : i+u], byteorder='little'))

    return l


def s_generator():
    p = 0xB7E1
    q = 0x9E37

    s = [p]

    for i in range(1, 2 * (r + 1)):
        s.append((s[i-1] + q) % (2 ** w))

    return s


def rotate_left(value, n):
```

```
    w2 = w // 2
    n = n % w2
    return ((value << n) & (2 ** w2 - 1)) | ((value & (2 ** w2 - 1)) >> (w2 - n))


def rotate_right(value, n):
    w2 = w // 2
    n = n % w2
    return ((value & (2 ** w2 - 1)) >> n) | (value << (w2 - n) & (2 ** w2 - 1))


def lsMixer(l, s):
    a, b = 0, 0
    i, j = 0, 0
    t = 2 * (r + 1)
    c = len(l)

    tc_max = max(t, c)

    for nei in range(3*tc_max):

        a = s[i] = rotate_left(s[i] + a + b, 3)
        b = l[j] = rotate_left(l[j] + a + b, a + b)
        i = (i + 1) % t
        j = (j + 1) % c

    return s


def shfr(lsmixed, data):
    w2 = w // 2
    b = w // 8
    module = 2 ** w2

    A = int.from_bytes(data[:(b//2)], byteorder='little')
    B = int.from_bytes(data[(b//2):], byteorder='little')

    A = (A + lsmixed[0]) % module
    B = (B + lsmixed[1]) % module

    for i in range(1, r+1):
        A = (rotate_left((A^B), B) + s[2 * i]) % module
        B = (rotate_left((A^B), A) + s[2 * i + 1]) % module

    return A.to_bytes(b // 2, byteorder='little') + B.to_bytes(b // 2, byteorder='little')


def le_shfr(codew, data):
    w2 = w // 2
    b = w // 8
    module = 2 ** w2
    A = int.from_bytes(data[:(b//2)], byteorder='little')
    B = int.from_bytes(data[(b//2):], byteorder='little')

    for i in range(r, 0, -1):
        B = rotate_right(B - codew[2 * i + 1], A)^A
        A = rotate_right(A - codew[2 * i], B)^B
```

```
    B = (B - codew[1]) % module
    A = (A - codew[0]) % module

    return A.to_bytes(b // 2, byteorder='little') + B.to_bytes(b // 2, byteorder='little')


def data_extention(data, keyls):
    w2 = w // 2
    b = w // 8

    while len(data) % b:
        data += b'\x00'

    res = b''
    last_shfrd = init_vect
    res += last_shfrd
    print("started")

    for i in range((len(data) // b)):
        ndata = data[b * i : b * (i + 1)]
        nblock = bytes([x^y for x, y in zip(last_shfrd, ndata)])
        last_shfrd = shfr(keyls, nblock)
        res += last_shfrd

    return res


def decrypt(data, keyls):
    w2 = w // 2
    b = w // 8

    last_shfrd = data[:b]
    res = b''
    ndata = data[b : b * 2]
    for i in range(2, len(data) // b + 1):

        unshfr = le_shfr(keyls, ndata)

        unshfr = bytes([x^y for x, y in zip(last_shfrd, unshfr)])
        last_shfrd = ndata
        ndata = data[b * i : b * (i + 1)]
        if not ndata:
            unshfr = unshfr.rstrip(b'\x00')
        res += unshfr

    return res


print("1. Code text\n2. Code file\n3. Decode text\n4. Decode file")
choise = int(input())

print("Input key:")
string = input()

hashed = MDFIVE(string)
hashed = hashed[16:]
hashed = hashed.encode('utf-8')
l = key_to_l(hashed)
```

```
s = s_generator()
lsmixed = lsMixer(l, s)

if (choise == 1):
    print("Input text: ")
    input = (input()).encode('utf-8')

    res = data_extention(input, lsmixed)
    with open("1.txt", 'wb') as file:
        file.write(res)

elif (choise == 2):
    print("Input file name:")
    Arkasha = input()

    with open(Arkasha, 'rb') as file:
        file_contents = file.read()

    res = data_extention(file_contents, lsmixed)

    with open(("code_" + Arkasha), 'wb') as file:
        file.write(res)

elif (choise == 3):
    with open("1.txt", 'rb') as file:
        file_contents = file.read()

    res = decrypt(file_contents, lsmixed)

    with open("2.txt", 'wb') as file:
        file.write(res)

elif (choise == 4):
    print("Input file name:")
    Arkasha = input()

    with open(Arkasha, 'rb') as file:
        file_contents = file.read()

    res = decrypt(file_contents, lsmixed)
    with open("uncode_" + Arkasha[5:], 'wb') as file:
        file.write(res)

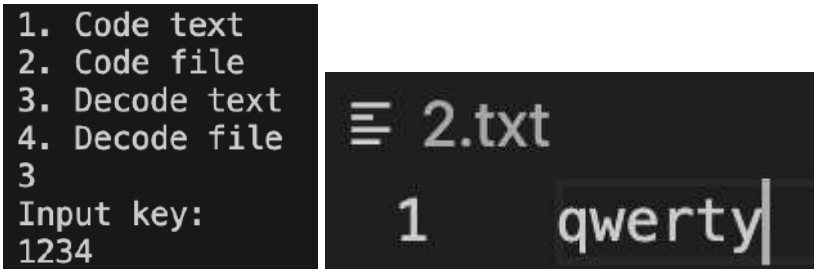
else:
    print("Nu i ladno.")
```

Результати виконання програми:

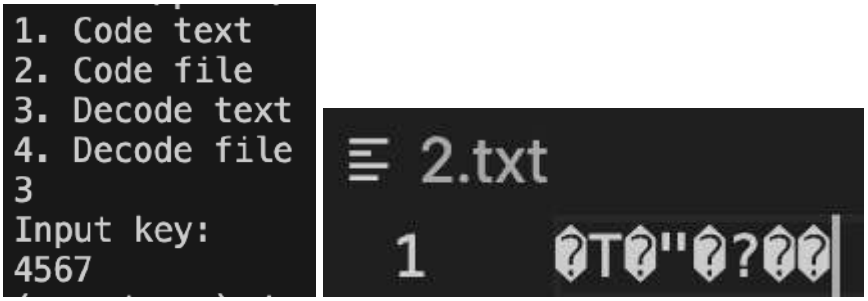
А) шифрування тексту



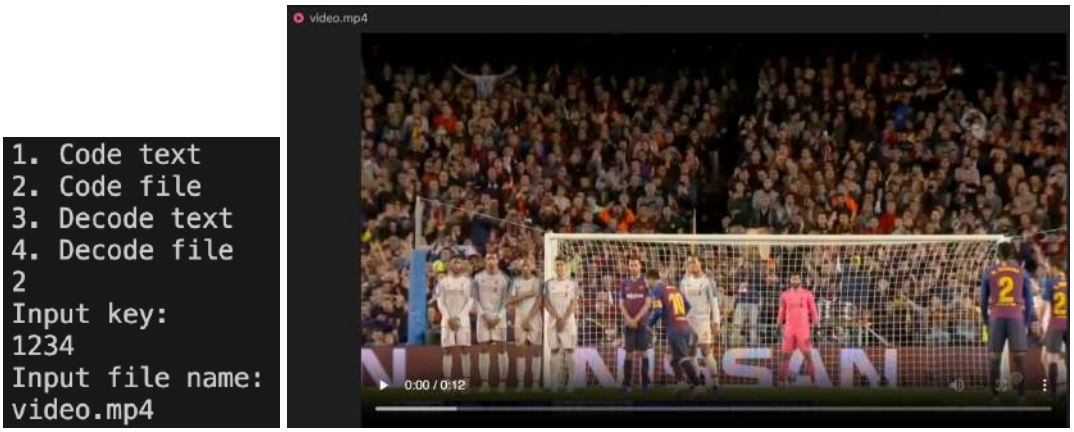
B) дешифрування з правильним ключем

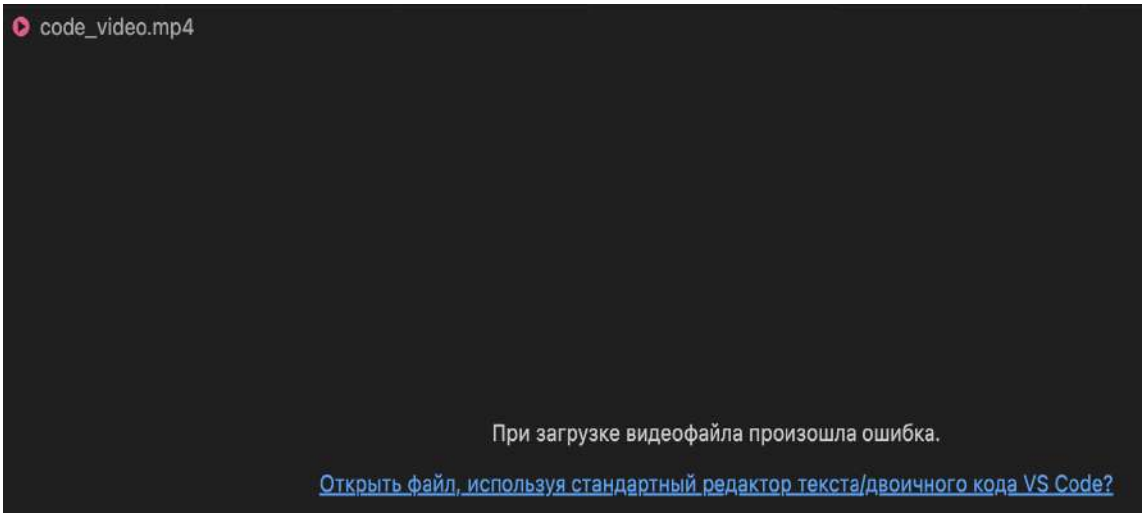


C) дешифрування з неправильним ключем

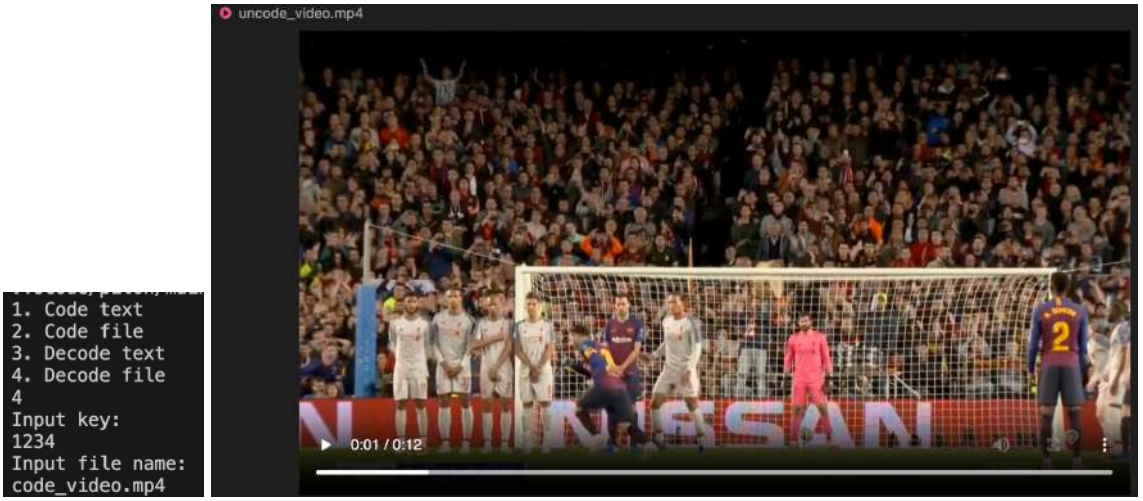


D) шифрування файлу





Е) дешифрування з правильним ключем



Висновок: ознайомився з методами криптографічного забезпечення конфіденційності інформації, навчився створювати комплексні програмні продукти для захисту інформації з використанням алгоритмів симетричного шифрування, хешування та генераторів псевдовипадкових чисел.